# PiFace Attendance System

## Complete Technical Build Specification for Claude Code

**Version 2.0 | Raspberry Pi 5 | Single Camera | Self-Contained | Zero Internet Hardened Edition -- All showstoppers and critical flaws from v1.0 audit resolved**

---

## Table of Contents


1. System Overview
2. Hardware Specification
3. System Architecture
4. Database Schema
5. Face Recognition Engine
6. Attendance Logic
7. LED Controller
8. WiFi Hotspot Configuration
9. Security Architecture
10. Web Dashboard -- All Pages
11. Backend API Endpoints
12. Auto-Generated Daily Reports
13. Pi Setup -- install.sh
14. Build Phases for Claude Code
15. Critical Rules for Claude Code
16. Complete Dependency Lists
17. Final Build Checklist


---

# 1. System Overview

PiFace is a fully self-contained, AI-powered office attendance monitoring system running entirely on a Raspberry Pi 5. It requires no internet connection, no cloud, no external IT infrastructure, and no technical expertise from the client. The client plugs it in and it works -- forever.

## 1.1 What The System Does

- Monitors a single office entrance camera continuously
- Recognises enrolled employees in real-time using InsightFace AI
- Logs every entry and exit event with timestamps
- Derives check-in, check-out, and break periods from the event log at summary time
- Automatically detects and tracks unknown visitors as `Unknown 1`, `Unknown 2` ... `Unknown N`
- Allows admin to rename any Unknown to a real person -- retroactively updating all records
- Broadcasts its own WiFi hotspot so any phone or laptop can access the web dashboard over HTTPS
- Controls two GPIO LEDs -- green for entry, red for exit -- completely independent of the UI

- Auto-generates daily attendance reports and stores them, available for download anytime
- Supports leave management and holiday calendars for accurate reporting
- All three core processes (face engine, web server, LED controller) run independently forever

## 1.2 What The System Does NOT Do

- Does NOT require internet connection at any point after initial setup
- Does NOT use any cloud service or external API
- Does NOT require client to have any technical knowledge
- Does NOT need a monitor, keyboard, or mouse attached to the Pi
- Does NOT stop working if the web dashboard is closed or nobody is connected

---

# 2. Hardware Specification

## 2.1 Required Components

| Component | Specification | Purpose | Approx Cost (INR) |
|---|---|---|---|
| Raspberry Pi 5 | 8GB RAM model | Main processing unit | 8,000 |
| **Raspberry Pi Active Cooler** | **Official fan + heatsink** | **MANDATORY -- prevents thermal throttling during 24/7 AI inference** | **300** |
| MicroSD Card | **Samsung PRO Endurance 64GB** | OS and data storage. Surveillance-grade for write endurance | 1,200 |
| Pi Power Supply | Official 27W USB-C | Stable power delivery | 800 |
| USB Webcam | 1080p, wide angle preferred (e.g. Logitech C270) | Face detection at entrance | 1,500 - 3,000 |
| **DS3231 RTC Module** | **I2C battery-backed real-time clock** | **Accurate timekeeping without internet** | **200** |
| Green LED | 5mm standard, 3.3V compatible | Entry confirmed indicator | 10 |
| Red LED | 5mm standard, 3.3V compatible | Exit confirmed indicator | 10 |
| 2x 220ohm Resistors | Through-hole | LED current limiting | 5 |
| Jumper wires | Female-to-male | GPIO connections | 50 |
| Project enclosure | Ventilated plastic box, 15x10x5cm | Housing the Pi (MUST have ventilation holes for active cooler) | 300 |

| Component | Specification | Purpose | Approx Cost (INR) |
|---|---|---|---|
| **USB WiFi Adapter (OPTIONAL)** | **RTL8812AU chipset** | **Only needed for true simultaneous hotspot + office WiFi bridge mode** | **800** |

**Total estimated cost: ~12,500 - 14,500 INR**

## 2.2 GPIO Wiring

```
Pi 5 GPIO Header (RP1 I/O Controller):

  PIN 11  (GPIO 17) --[ 220ohm ]--[ GREEN LED + ]--[ GND (PIN 9)  ]
  PIN 13  (GPIO 27) --[ 220ohm ]--[ RED LED   + ]--[ GND (PIN 14) ]

Note: Always use resistors. Never connect LED directly to GPIO.
Note: Pi 5 uses RP1 chip. Use gpiozero library (NOT RPi.GPIO, which is
broken on Pi 5).
```

## 2.3 RTC Module Wiring (DS3231)

```
DS3231       Pi 5
  VCC   -->  PIN 1  (3.3V)
  GND   -->  PIN 6  (GND)
  SDA   -->  PIN 3  (GPIO 2 / SDA)
  SCL   -->  PIN 5  (GPIO 3 / SCL)
```

## 2.4 Camera Placement

- USB webcam recommended for simplicity (Pi Camera Module 3 requires picamera2 + libcamera complexity)
- Mounted at approximately 1.2-1.5 metres height
- Facing the direction of incoming movement for clearest face capture
- Angled slightly downward -- approximately 15-20 degrees
- Ensure good lighting on the face side -- not backlit from a window
- If facing a glass door, angle camera to avoid reflections and outdoor passers-by

---

# 3. System Architecture

## 3.1 Three Independent Processes (Run as `piface` User, NOT Root)

**Process 1 -- Face Engine (`core/face_engine.py`)**

- Reads camera feed continuously via OpenCV (USB) or picamera2 (Pi Camera)
- Auto-detects camera type at startup
- Applies CLAHE preprocessing for lighting robustness
- Processes every 3rd frame for detection, uses centroid tracking between detection frames
- Runs InsightFace detection + recognition on detection frames
- Compares detected faces against enrolled embeddings (cached in memory)
- Uses IoU-based multi-object tracking with persistent track IDs
- Tracks person movement using time-windowed position vectors (1.5 seconds)
- Compares movement to calibrated IN/OUT vectors using cosine similarity
- Writes attendance events to SQLite database (WAL mode)
- Fires LED events via Unix domain socket
- Sends systemd watchdog pings every 10 seconds
- Runs 24/7 regardless of whether web dashboard is open
- Pinned to CPU cores 0-1 via systemd CPUAffinity

**Process 2 -- Web Server (`backend/main.py`)**

- FastAPI application serving REST API over Unix socket
- Nginx reverse-proxies from HTTPS (port 443) to FastAPI Unix socket
- Provides all dashboard endpoints: employees, attendance, reports, settings, leave, holidays
- Reads from and writes to the same SQLite database (WAL mode enables concurrent access)
- Can crash and auto-restart without affecting face engine or LEDs
- Pinned to CPU core 2 via systemd CPUAffinity

**Process 3 -- LED Controller (`core/led_controller.py`)**

- Listens on a Unix domain socket at `/run/piface/leds.sock` for signals from face engine
- Controls GPIO 17 (green) and GPIO 27 (red) via **gpiozero** library (Pi 5 compatible)
- Executes LED patterns independently -- no web dependency
- Handles system boot blink sequence on startup
- Cleans up stale socket file before bind
- Sends systemd watchdog pings
- Pinned to CPU core 3 via systemd CPUAffinity

## 3.2 Data Flow

```
Camera Frame (30 FPS from camera)
    |
    v
Frame Grabber (separate thread, always grabs latest frame, discards stale)
    |
    v
Every 3rd frame --> Full InsightFace Pipeline
Other frames ----> Centroid-only tracking (lightweight, <1ms)
    |
    v
CLAHE Preprocessing (adaptive histogram equalization on LAB L-channel,
~5ms)
    |
```

```
    v
InsightFace Detection (RetinaFace -- is there a face? ~80-120ms)
    |
    +-- NO FACE --> skip
    |
    +-- FACE FOUND
           |
           v
       Face Recognition (ArcFace 512D embedding, ~50-80ms per face)
           |
           +-- MATCH (cosine similarity > 0.6, margin > 0.08 vs second-
best)
           |      |
           |      v
           |  Direction Check (cosine similarity vs calibrated vector)
           |      |
           |      +-- IN  (score > 0.7)  --> log IN event + fire GREEN LED
           |      +-- OUT (score > 0.7)  --> log OUT event + fire RED LED
           |      +-- NEITHER            --> stationary, no event
           |
           +-- NO MATCH (below 0.6 or margin < 0.08)
                  |
                  v
              Check against existing unknowns (similarity > 0.55)
              If truly new: assign Unknown N, save snapshot
              Track IN/OUT same as known employees
              Fire ALTERNATE LED (green-red blink)
```

## 3.3 Project Directory Structure

```
piface/
+-- core/
|   +-- face_engine.py          # InsightFace recognition engine (main
loop)
|   +-- tracker.py              # IoU-based multi-object tracking +
direction detection
|   +-- camera.py               # Camera auto-detection (USB/Pi), CLAHE,
frame management
|   +-- led_controller.py       # gpiozero LED control (independent
process)
|   +-- event_bus.py            # Unix domain socket IPC between processes
|   +-- preprocessing.py        # CLAHE, blur detection, enrollment quality
gates
|
+-- backend/
|   +-- main.py                 # FastAPI entry point with security
middleware
|   +-- database.py             # SQLAlchemy setup, WAL mode, FK
enforcement, migrations
|   +-- models.py               # DB table models with indexes and
constraints
```

```
|   +-- schemas.py              # Pydantic request/response schemas
|   +-- security.py             # JWT, CSRF, rate limiting, TLS
|   +-- routes/
|       +-- employees.py        # Add/edit/delete employees + enrollment
|       +-- attendance.py       # Event log, manual corrections, today
summary
|       +-- unknowns.py         # Unknown persons + rename
|       +-- reports.py          # Daily/weekly/monthly reports
|       +-- calibration.py      # IN/OUT direction vector endpoints
|       +-- settings.py         # System settings
|       +-- auth.py             # Login/logout with rate limiting
|       +-- leave.py            # Leave records + holidays
|       +-- system.py           # Health, backup, restart, status
|
+-- frontend/                   # React app (built to frontend/dist/)
|   +-- src/
|   |   +-- pages/              # Lazy-loaded route components
|   |   |   +-- Today.jsx       # Live attendance dashboard
|   |   |   +-- Employees.jsx   # Employee management
|   |   |   +-- Unknowns.jsx    # Unknown persons page
|   |   |   +-- Reports.jsx     # Reports + download (lazy-loads recharts)
|   |   |   +-- Settings.jsx    # Settings page
|   |   |   +-- Setup/          # First-time setup wizard (4 steps)
|   |   |   +-- Login.jsx       # Admin login
|   |   |   +-- Leave.jsx       # Leave management
|   |   +-- components/
|   |   |   +-- Calibration.jsx # Canvas overlay with Pointer Events API
|   |   |   +-- LiveFeed.jsx    # MJPEG stream (off by default, auto-off
timer)
|   |   |   +-- Layout/         # AppShell, BottomNav (mobile), Sidebar
(desktop)
|   |   |   +-- ConnectionBanner.jsx  # Offline/reconnecting indicator
|   |   +-- hooks/
|   |   |   +-- useSSE.js       # SSE with auto-reconnect + backoff
|   |   |   +-- useAuth.js      # Auth context + 401 interceptor
|   |   +-- api/
|   |       +-- client.js       # axios with withCredentials, interceptors
|
+-- setup/
|   +-- install.sh              # Full Pi setup script (with error
handling)
|   +-- hostapd.conf            # WiFi hotspot configuration (CCMP only,
802.11n)
|   +-- dnsmasq.conf            # DHCP + captive portal
|   +-- nginx.conf              # HTTPS reverse proxy with security headers
|   +-- piface-engine.service   # systemd: face engine (cores 0-1,
watchdog)
|   +-- piface-web.service      # systemd: web server (core 2, watchdog)
|   +-- piface-leds.service     # systemd: LED controller (core 3,
watchdog)
|   +-- logrotate.conf          # Log rotation for /var/log/piface/
|   +-- iptables.rules          # Firewall rules
|
+-- database/
```

```
|   +-- attendance.db           # SQLite database (WAL mode, auto-created)
|
+-- backups/                    # Nightly database backups
|
+-- reports/                    # Auto-generated daily reports (PDF +
Excel)
|   +-- YYYY-MM-DD/
|
+-- snapshots/                  # Unknown person face snapshots (UUID
filenames)
|
+-- models/                     # InsightFace model files (Buffalo_S, pre-
bundled)
|
+-- certs/                      # Self-signed TLS certificate + key
```

---

# 4. Database Schema

## 4.1 Initialization PRAGMAs (MANDATORY on every connection)

```
PRAGMA journal_mode=WAL;            -- concurrent reads + writes
PRAGMA synchronous=NORMAL;          -- balance safety vs performance
PRAGMA busy_timeout=5000;           -- 5s retry on write contention
PRAGMA foreign_keys=ON;             -- enforce referential integrity
PRAGMA wal_autocheckpoint=1000;     -- prevent unbounded WAL growth
```

In SQLAlchemy, enforce via engine event listener:

```
@event.listens_for(engine, "connect")
def set_sqlite_pragma(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA journal_mode=WAL")
    cursor.execute("PRAGMA synchronous=NORMAL")
    cursor.execute("PRAGMA busy_timeout=5000")
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()
```

On startup, run `PRAGMA quick_check` to detect corruption early.

## 4.2 `persons` Table

| Column | Type | Description |
|---|---|---|
| id | INTEGER PRIMARY KEY | Auto-increment ID |

| Column | Type | Description |
|---|---|---|
| name | TEXT NOT NULL | Full name (or 'Unknown N' if unidentified) |
| employee_id | TEXT UNIQUE | Employee ID (NULL for unknowns) |
| department | TEXT | Department name |
| job_title | TEXT | Job title / designation |
| phone | TEXT | Phone number (optional) |
| face_embedding | BLOB NOT NULL CHECK(LENGTH(face_embedding) = 2048) | 512D float32 array (512 x 4 bytes = 2048 bytes) |
| face_image | TEXT | Path to enrollment face snapshot |
| is_unknown | BOOLEAN DEFAULT FALSE | True if auto-assigned Unknown N |
| unknown_index | INTEGER | N value for Unknown N (NULL if named) |
| enrolled_at | DATETIME | Enrollment timestamp |
| is_active | BOOLEAN DEFAULT TRUE | Soft delete flag |

**Indexes:**

```
CREATE INDEX idx_persons_active ON persons(is_active);
CREATE INDEX idx_persons_unknown ON persons(is_unknown);
```

## 4.3 `attendance_events` Table

| Column | Type | Description |
|---|---|---|
| id | INTEGER PRIMARY KEY | Auto-increment |
| person_id | INTEGER NOT NULL REFERENCES persons(id) ON DELETE RESTRICT | Who was detected |
| event_type | TEXT NOT NULL | IN / OUT (simple -- derived types computed at summary time) |
| timestamp | DATETIME NOT NULL | Exact time of event |
| confidence | REAL | Face match confidence score (0.0 - 1.0) |
| snapshot_path | TEXT | Path to face snapshot at time of detection |
| direction_vector | TEXT | JSON [dx,dy] -- movement vector at detection |

| Column | Type | Description |
|---|---|---|
| date | DATE GENERATED ALWAYS AS (DATE(timestamp)) STORED | Auto-computed from timestamp |
| is_manual | BOOLEAN DEFAULT FALSE | True if manually entered/corrected by admin |
| corrected_by | TEXT | Admin username who made the correction |

**Indexes:**

```
CREATE INDEX idx_events_person_date ON attendance_events(person_id, date);
CREATE INDEX idx_events_date ON attendance_events(date);
```

## 4.4 `daily_summary` Table

| Column | Type | Description |
|---|---|---|
| id | INTEGER PRIMARY KEY | Auto-increment |
| person_id | INTEGER NOT NULL REFERENCES persons(id) ON DELETE RESTRICT | Employee reference |
| date | DATE NOT NULL | Date of summary |
| first_in_time | DATETIME | First IN event of day |
| last_out_time | DATETIME | Last OUT event of day |
| total_in_out_events | INTEGER | Number of IN/OUT transitions |
| total_hours_worked | REAL | Total hours (last OUT - first IN - breaks) |
| longest_break_minutes | INTEGER | Longest continuous absence (likely lunch) |
| total_break_minutes | INTEGER | Sum of all breaks |
| is_late | BOOLEAN | Arrived after configured shift start |
| is_early_leave | BOOLEAN | Left before configured shift end |
| overtime_minutes | INTEGER | Minutes worked beyond shift end |
| status | TEXT | PRESENT / ABSENT / HALF_DAY / ON_LEAVE / HOLIDAY |

**Constraints:**

```
UNIQUE(person_id, date)
```

## 4.5 `system_settings` Table

| Column | Type | Description |
|---|---|---|
| key | TEXT PRIMARY KEY | Setting key |
| value | TEXT | Setting value (JSON serialised) |
| updated_at | DATETIME | Last modified time |

**Settings keys:** `company_name`, `shift_start`, `shift_end`, `late_threshold_minutes`, `max_break_minutes`, `wifi_ssid`, `wifi_password`, `in_vector`, `out_vector`, `setup_complete`, `unknown_counter`, `face_threshold`, `unknown_threshold`, `snapshot_retention_days`, `report_retention_days`, `weekend_days`

**Note:** Admin credentials are stored in `auth_users` table, NOT here.

## 4.6 `auth_users` Table

| Column | Type | Description |
|---|---|---|
| id | INTEGER PRIMARY KEY | Auto-increment |
| username | TEXT UNIQUE NOT NULL | Admin username |
| password_hash | TEXT NOT NULL | bcrypt hash |
| role | TEXT DEFAULT 'admin' | User role |
| created_at | DATETIME DEFAULT CURRENT_TIMESTAMP | Creation time |
| last_login | DATETIME | Last login timestamp |

## 4.7 `audit_log` Table

| Column | Type | Description |
|---|---|---|
| id | INTEGER PRIMARY KEY | Auto-increment |
| action | TEXT NOT NULL | DELETE_EMPLOYEE, RENAME_UNKNOWN, CHANGE_SETTINGS, etc. |
| target_table | TEXT | Affected table |
| target_id | INTEGER | Affected row ID |
| old_value | TEXT | JSON of previous state |
| new_value | TEXT | JSON of new state |
| performed_by | TEXT | Admin username |

| Column | Type | Description |
|---|---|---|
| performed_at | DATETIME DEFAULT CURRENT_TIMESTAMP | When |

### 4.8 `leave_records` Table

| Column | Type | Description |
|---|---|---|
| id | INTEGER PRIMARY KEY | Auto-increment |
| person_id | INTEGER NOT NULL REFERENCES persons(id) | Employee |
| date | DATE NOT NULL | Date of leave |
| leave_type | TEXT NOT NULL | VACATION / SICK / PERSONAL / WFH |
| note | TEXT | Optional note |

**Constraints:** `UNIQUE(person_id, date)`

### 4.9 `holidays` Table

| Column | Type | Description |
|---|---|---|
| id | INTEGER PRIMARY KEY | Auto-increment |
| date | DATE UNIQUE NOT NULL | Holiday date |
| name | TEXT NOT NULL | Holiday name |

### 4.10 `schema_migrations` Table

| Column | Type | Description |
|---|---|---|
| version | INTEGER PRIMARY KEY | Migration version number |
| applied_at | DATETIME DEFAULT CURRENT_TIMESTAMP | When applied |
| description | TEXT | What changed |

## 5. Face Recognition Engine

### 5.1 Model

- **Library:** InsightFace (`pip install insightface`)
- **Model:** Buffalo_S -- optimised for ARM64, best speed/accuracy on Pi 5
- **Model files:** Pre-bundled in `models/buffalo_s/` -- NOT downloaded at runtime
- **Embedding size:** 512 dimensions per face (2048 bytes as float32)
- **Detection backend:** SCRFD-500M (RetinaFace variant)
- **Recognition backend:** MobileFaceNet (ArcFace)

- **Known face threshold:** 0.6 (configurable 0.4-0.8 in settings). Industry standard range.
- **Top-2 margin check:** Best match must exceed second-best by at least 0.08 to commit
- **Unknown matching threshold:** 0.55 (higher than v1's 0.45 to prevent false merging)

## 5.2 CLAHE Preprocessing (NEW)

Before every detection frame, apply adaptive histogram equalization:

```
1. Convert BGR frame to LAB color space
2. Apply CLAHE to L channel (clipLimit=2.0, tileGridSize=8x8)
3. Merge back to LAB, convert to BGR
4. Pass preprocessed frame to InsightFace
```

Cost: ~5ms per frame. Dramatically improves detection in backlit doorways and uneven lighting.

## 5.3 Frame Processing Strategy

```
Camera delivers 30 FPS.
InsightFace processes ~5-7 FPS on Pi 5.

Strategy:
- Frame grabber thread: continuously grabs latest frame, discards buffer
  (set cv2.CAP_PROP_BUFFERSIZE = 1)
- Processing loop:
  - Every 3rd frame: run full InsightFace (detection + recognition)
  - Other frames: update centroid positions using IoU matching (< 1ms)
- Effective detection rate: ~10 FPS
- Effective tracking rate: ~30 FPS (centroid only)
```

## 5.4 Enrollment Process

```
If VIDEO uploaded:
  1. Extract all frames with OpenCV
  2. QUALITY GATES:
     a. Reject if video shorter than 2 seconds
     b. For each frame, run blur detection (Laplacian variance)
     c. Reject frames with Laplacian variance < 100 (too blurry)
     d. Reject frames where detected face bbox < 112x112 pixels (too small)
     e. Reject frames with more than one face detected
  3. Keep top 10 sharpest frames that pass all gates
  4. Reject enrollment if fewer than 5 frames pass (video quality too poor)
  5. Run InsightFace on each kept frame, extract 512D embedding
  6. Check inter-similarity: average pairwise cosine similarity among the
10 embeddings
     must be > 0.7 (consistent face captures). If not, warn admin.
  7. Average all embeddings --> single enrollment embedding
  8. Pre-normalize to unit vector (makes cosine similarity = dot product at
```

```
match time)

If PHOTO uploaded:
  1. QUALITY GATES:
     a. Reject if Laplacian variance < 100 (blurry)
     b. Reject if face bbox < 112x112 pixels
     c. Reject if more than one face detected (or require user to select)
     d. Reject if det_score < 0.5 (low-quality detection)
  2. Run InsightFace, extract 512D embedding
  3. Pre-normalize to unit vector
  4. Warn admin: "Single photo enrollment is less reliable than video"

Store embedding as BLOB in persons.face_embedding
Store best frame as face_image path (UUID filename)
Send REFRESH signal to face engine via Unix socket to reload cache
immediately
```

## 5.5 Real-Time Recognition

```
Every detection frame (every 3rd frame):
  1. CLAHE preprocess
  2. InsightFace.get() --> list of detected faces
  3. For each detected face:
     a. Extract 512D embedding, normalize to unit vector
     b. Batch cosine similarity: embeddings_matrix @ query_embedding (numpy
vectorized)
     c. Find top-2 matches
     d. If best_score > 0.6 AND (best_score - second_score) > 0.08:
        --> KNOWN person (assign identity to track)
     e. If best_score > 0.6 BUT margin < 0.08:
        --> AMBIGUOUS, do not commit identity, continue tracking
     f. If best_score < 0.6:
        --> UNKNOWN (check against existing unknowns at threshold 0.55)
  4. Assign face identity to the persistent track ID (not to the raw
detection)

Embedding cache:
  - All enrolled embeddings loaded into a numpy matrix at startup
  - Refreshed every 30 seconds OR immediately on REFRESH signal via Unix
socket
  - Pre-normalized at load time for fast dot-product similarity
```

## 5.6 Multi-Object Tracking (IoU-Based)

```
Each tracked person has a persistent track_id (integer, incremented per new
track).

On every frame (including non-detection frames):
  1. Predict next position of existing tracks (simple linear velocity
```

```
model)
  2. If detection frame: compute IoU between predicted boxes and detected
boxes
  3. Use Hungarian algorithm (scipy.optimize.linear_sum_assignment) for
assignment
  4. Matched tracks: update position, keep identity
  5. Unmatched detections: create new track
  6. Unmatched tracks (no detection for 15 frames): mark track as lost

This prevents identity swaps when two people cross paths or walk side by
side.
```

## 5.7 IN/OUT Direction Detection -- Cosine Vector Method

```
Admin draws IN arrow:  point A(x1,y1) -> point B(x2,y2)
Stored as IN vector:   [x2-x1, y2-y1]  normalized to unit vector

Admin draws OUT arrow: point C(x3,y3) -> point D(x4,y4)
Stored as OUT vector:  [x4-x3, y4-y3]  normalized to unit vector

Direction detection per track:
  - Collect centroid positions over a 1.5-second sliding window
  - Compute movement vector: [latest_x - earliest_x, latest_y - earliest_y]
  - Minimum movement: 0.3 * face_bbox_width (relative, not fixed pixels)
  - If movement below minimum: person is stationary, no event
  - Cosine similarity with IN vector  --> in_score
  - Cosine similarity with OUT vector --> out_score
  - in_score  > 0.7 --> log as IN event
  - out_score > 0.7 --> log as OUT event
  - Neither above 0.7 --> no event
  - Cooldown: same person cannot trigger another event for 45 seconds
    (reduced from 60s -- allows for "forgot keys" scenarios)
```

## 5.8 Unknown Person Handling

```
Face detected, no match above threshold:

1. Check if this embedding matches any existing unknown
   (similarity > 0.55)
   --> If matches: same unknown, continue tracking under that track

2. Throttle check: if more than 20 new unknowns created in the last hour,
   flag camera misconfiguration alert (LED pattern + dashboard warning).
   Do not create new unknowns until admin acknowledges.

3. If truly new face:
   --> Increment unknown_counter atomically in DB
   --> Create persons record: name='Unknown N', is_unknown=TRUE
   --> Save face snapshot to snapshots/{uuid}.jpg (NOT predictable
```

```
filenames)
   --> Begin tracking this person

4. Log their IN/OUT events same as known employees

Admin renames Unknown N:
   --> UPDATE persons SET name='Real Name', is_unknown=FALSE, ...
   --> All past attendance_events for that person_id now show real name
   --> Log action in audit_log
```

---

# 6. Attendance Logic (Event-Log Model)

## 6.1 Event Recording

Every detected direction event is recorded as a simple IN or OUT in `attendance_events`. There is no rigid state machine. The face engine records what it observes:

```
Person detected moving IN direction  --> INSERT (person_id, 'IN',
timestamp, ...)
Person detected moving OUT direction --> INSERT (person_id, 'OUT',
timestamp, ...)
```

**Duplicate prevention:**

- 45-second cooldown per person per direction (same person, same direction)
- Different direction resets the cooldown (allows quick IN then OUT for "forgot keys")

## 6.2 Daily Summary Computation (Derived, Not Rigid)

Runs automatically at 23:59:00 via APScheduler, AND on boot for any missing days:

```
For each person with events on date D:

  events = SELECT * FROM attendance_events
           WHERE person_id=? AND date=? ORDER BY timestamp ASC

  first_in  = first IN event timestamp
  last_out  = last OUT event timestamp

  -- Compute breaks: every OUT followed by a subsequent IN is a break
  breaks = []
  for each consecutive (OUT, IN) pair in events:
      break_duration = IN.timestamp - OUT.timestamp
      breaks.append(break_duration)

  total_break_minutes = sum(breaks)
```

```
  longest_break_minutes = max(breaks) if breaks else 0  -- likely lunch

  total_hours = (last_out - first_in) - total_break_minutes
                (if no last_out, total_hours = NULL = still IN)

  is_late = first_in > (shift_start + late_threshold_minutes)
  is_early_leave = last_out < shift_end (if last_out exists)
  overtime = max(0, last_out - shift_end) in minutes

  -- Status determination:
  IF person has a leave_record for date D:
      status = ON_LEAVE
  ELIF date D is in holidays table:
      status = HOLIDAY
  ELIF date D is a configured weekend day:
      status = HOLIDAY
  ELIF no events at all:
      status = ABSENT
  ELIF total_hours < shift_hours / 2:
      status = HALF_DAY
  ELSE:
      status = PRESENT

  UPSERT INTO daily_summary (person_id, date, ...)
```

## 6.3 Cross-Midnight Handling

```
When an OUT event is detected and the person has no IN event today:
  1. Check previous day: does this person have an open session (IN with no
final OUT)?
  2. If yes: attribute this OUT to the previous day's session
     - Write the OUT event with today's timestamp but link to previous
day's summary
     - Recompute previous day's daily_summary with this new OUT time
  3. If no: ignore (spurious detection, person walking past)
```

## 6.4 State Reconstruction on Restart

```
On face engine startup:
  1. For each active person, query today's events from DB
  2. Reconstruct current state:
     - If last event is IN: person is currently IN (don't re-log IN on
first detection)
     - If last event is OUT: person is currently OUT
     - If no events: no prior state
  3. Apply cooldown: if last event was within 45 seconds, maintain cooldown
```

### 6.5 Manual Corrections

```
Admin can via the dashboard:
  - ADD a missing event (e.g., person forgot to walk past camera)
  - EDIT an incorrect event (change type or timestamp)
  - DELETE an erroneous event (false detection)

All manual changes:
  - Set is_manual=TRUE, corrected_by=admin_username
  - Log in audit_log with old and new values
  - Trigger recomputation of daily_summary for affected date
```

---

## 7. LED Controller

### 7.1 LED Patterns -- Full Reference

| Event | Pattern | Duration | GPIO |
|---|---|---|---|
| System boot complete | 5 rapid green blinks | 2 seconds total | GPIO 17 (green) |
| Known person -- IN event | Green solid ON | 3 seconds | GPIO 17 (green) |
| Known person -- OUT event | Red solid ON | 3 seconds | GPIO 27 (red) |
| Unknown person detected | Green-Red alternate blink | 2 seconds | Both |
| Camera disconnected | Red slow blink (every 2s) | Continuous until reconnected | GPIO 27 (red) |
| Camera reconnected | 3 green blinks | 1.5 seconds | GPIO 17 (green) |
| System error / crash | Red double blink (every 3s) | Continuous until fixed | GPIO 27 (red) |
| Unknown throttle alert | Rapid red blinks | 5 seconds | GPIO 27 (red) |
| Low disk space warning | Red-green alternate (slow) | 5 seconds, every 10 min | Both |
| Face engine starting up | Both LEDs on steady | During startup only | Both |

### 7.2 Implementation Requirements

```
# Use gpiozero (NOT RPi.GPIO -- broken on Pi 5)
from gpiozero import LED
from signal import pause

green = LED(17)
red = LED(27)

# Patterns use gpiozero's built-in blink() -- non-blocking, thread-based
green.blink(on_time=0.2, off_time=0.2, n=5)  # boot sequence
```

- Socket file: `/run/piface/leds.sock` (tmpfs, auto-cleared on reboot)
- `ExecStartPre=-/bin/rm -f /run/piface/leds.sock` in service file
- `RuntimeDirectory=piface` in service file creates `/run/piface/` automatically
- SIGTERM handler: turn off all LEDs, close socket, exit cleanly
- Watchdog: send `sd_notify("WATCHDOG=1")` every 10 seconds

---

# 8. WiFi Hotspot Configuration

## 8.1 Stack

- **hostapd** -- creates and manages the WiFi access point
- **dnsmasq** -- DHCP server + captive portal DNS
- **Nginx** -- HTTPS reverse proxy to FastAPI, serves React frontend
- **NetworkManager** -- manages eth0 only. wlan0 is unmanaged (configured directly by hostapd)

## 8.2 Network Details

| Setting | Value | Notes |
|---|---|---|
| WiFi SSID | AttendanceSystem (default) | Admin can change in Settings |
| WiFi Password | Set during first-time wizard | Minimum 8 characters |
| WiFi Security | WPA2-PSK with AES/CCMP only | No TKIP (deprecated, insecure) |
| WiFi Mode | 802.11n on 2.4 GHz | ~150 Mbps vs 54 Mbps with 802.11g |
| WiFi Channel | Auto-select (ACS) | Picks least congested channel |
| Client isolation | Enabled | Clients cannot see each other |
| Max stations | 20 | Enforced in hostapd |
| Pi static IP | 192.168.4.1 | Never changes |
| DHCP range | 192.168.4.2 - 192.168.4.50 | 49 addresses, 1-hour lease |
| Web UI URL | https://192.168.4.1 | Self-signed certificate |
| Captive portal | Proper OS-specific detection | Works on Android, iOS, Windows |

## 8.3 hostapd.conf

```
interface=wlan0
driver=nl80211
ssid=AttendanceSystem
hw_mode=g
channel=0
ieee80211n=1
wmm_enabled=1
macaddr_acl=0
auth_algs=1
ignore_broadcast_ssid=0
wpa=2
wpa_passphrase=CHANGE_ON_SETUP
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
ap_isolate=1
max_num_sta=20
```

## 8.4 dnsmasq.conf

```
interface=wlan0
listen-address=192.168.4.1
dhcp-range=192.168.4.2,192.168.4.50,255.255.255.0,1h
domain=local
address=/#/192.168.4.1
```

## 8.5 Captive Portal Detection (Proper)

Nginx must serve correct responses for OS-specific captive portal probes:

```
# Android
location /generate_204 { return 302 https://192.168.4.1/; }
location /gen_204 { return 302 https://192.168.4.1/; }

# Apple
location /hotspot-detect.html {
    return 200 '<HTML><HEAD><TITLE>Success</TITLE></HEAD>
<BODY>Success</BODY></HTML>';
}

# Windows
location /connecttest.txt { return 200 'Microsoft Connect Test'; }
location /ncsi.txt { return 200 'Microsoft NCSI'; }

# Firefox
location /success.txt { return 200 'success'; }
```

## 8.6 WiFi Mode Switching

**Hotspot Mode (default):** Pi broadcasts its own WiFi. Fully isolated.

**Office WiFi Mode:** Pi joins existing office WiFi. Hotspot is disabled. Dashboard accessible at Pi's DHCP-assigned IP on the office network.

**Dual Mode (requires USB WiFi adapter):** onboard wlan0 runs hotspot, USB wlan1 joins office WiFi. Both work simultaneously. Only available if USB WiFi adapter is detected.

Mode switching is handled by the Settings page calling a backend endpoint that validates the config, creates a backup, rewrites config files, and restarts networking.

---

# 9. Security Architecture (NEW SECTION)

## 9.1 Transport Security

- **Self-signed TLS certificate** generated during `install.sh` (RSA 2048, 10-year validity)
- Stored at `/opt/piface/certs/piface.crt` and `/opt/piface/certs/piface.key`
- Nginx terminates TLS on port 443, proxies to FastAPI Unix socket
- Port 80 redirects to 443
- Uvicorn binds to Unix socket only (`/run/piface/web.sock`), NOT 0.0.0.0:8000

## 9.2 Authentication

- JWT tokens with 2-hour expiration, stored in httpOnly + Secure + SameSite=Strict cookies
- JWT signing secret: 256-bit random, generated per-device during install, stored at `/opt/piface/config/jwt_secret.key` (chmod 400)
- Login endpoint: `POST /api/auth/login` with bcrypt password verification
- Rate limiting: max 5 failed login attempts per IP per 5 minutes. After 10 consecutive failures, 15-minute global lockout
- Logout: server-side token blacklist (short-lived entries, auto-cleaned)

## 9.3 CSRF Protection

- All state-changing endpoints (POST, PUT, DELETE) require `X-CSRF-Token` header
- CSRF token generated per session, sent as a non-httpOnly cookie
- Frontend reads CSRF cookie and sends it as a header on every mutation

## 9.4 Firewall (iptables)

```
# Default policy: drop everything
iptables -P INPUT DROP
iptables -P FORWARD DROP

# Allow loopback
```

```
iptables -A INPUT -i lo -j ACCEPT

# Allow established connections
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Allow DHCP (UDP 67-68)
iptables -A INPUT -i wlan0 -p udp --dport 67:68 -j ACCEPT

# Allow DNS (UDP/TCP 53)
iptables -A INPUT -i wlan0 -p udp --dport 53 -j ACCEPT
iptables -A INPUT -i wlan0 -p tcp --dport 53 -j ACCEPT

# Allow HTTPS (TCP 443)
iptables -A INPUT -i wlan0 -p tcp --dport 443 -j ACCEPT

# Allow HTTP (TCP 80, for redirect to HTTPS)
iptables -A INPUT -i wlan0 -p tcp --dport 80 -j ACCEPT

# Drop everything else (SSH, port 8000, etc.)
```

### 9.5 Process Isolation

- Dedicated `piface` system user (no login shell)
- All three services run as `User=piface`, `Group=piface`
- `piface` user added to groups: `gpio`, `video`, `i2c`
- systemd sandboxing: `ProtectSystem=strict`, `ProtectHome=yes`, `NoNewPrivileges=true`
- `ReadWritePaths=/opt/piface/database /opt/piface/reports /opt/piface/snapshots /opt/piface/backups /var/log/piface /run/piface`

### 9.6 File Upload Security

- Never use user-supplied filenames. All uploads renamed to UUID
- Validate MIME type via magic bytes (python-magic), not just Content-Type header
- Max file size: 10MB photos, 50MB videos
- `client_max_body_size 50m` in Nginx
- Validate uploaded files can be decoded by OpenCV before storing
- Store uploads outside Nginx web root. Serve only through authenticated API endpoints.

### 9.7 Sensitive Endpoints

- Factory reset: requires admin password re-entry + 30-second confirmation window
- Service restart: whitelist-only helper script (`/opt/piface/helpers/restart_service.sh`) with sudoers entry for `piface` user
- WiFi config changes: template-based (web server writes JSON, helper script validates and applies)
- The web server process NEVER directly calls `systemctl` or writes to `/etc/`

---

## 10. Web Dashboard -- All Pages

---

## 10.1 Design Principles

- **Mobile-first:** Most users access from phones. Every page designed for 375px+ width first.
- **Responsive breakpoints:** phone (<640px), tablet (640-1024px), desktop (>1024px)
- **Touch-friendly:** All interactive elements minimum 44x44px tap target
- **Lazy-loaded routes:** Each page is a React.lazy() import with Suspense fallback
- **Offline-aware:** ConnectionBanner component shows WiFi status. Auto-retry on reconnect.
- **Accessible:** Color-coded statuses always have text labels. ARIA attributes on all interactive elements.

## 10.2 First-Time Setup Wizard *(shown only once)*

Triggered when `setup_complete = false`. Wizard blocks all other navigation.

- **Step 1 -- Company Info:** company name, admin username, admin password (min 8 chars)
- **Step 2 -- Working Hours:** shift start, shift end, late threshold, weekend days selection
- **Step 3 -- Direction Calibration:** live camera feed shown via canvas overlay. Admin draws IN arrow and OUT arrow using **Pointer Events API** (works on both touch and mouse). Touch-action: none on drawing surface. Coordinate normalization to 640x480.
- **Step 4 -- Completion:** `setup_complete = true`, redirect to Today page

Each step has its own URL segment (`/setup/company`, `/setup/hours`, etc.) for browser back-button support. `beforeunload` listener prevents accidental page close during setup.

## 10.3 Today Page

- Live list of all enrolled employees with current status:
  - Green dot + "IN" -- currently in office
  - Yellow dot + "Break" -- last event was OUT, but a subsequent IN is expected
  - Grey dot + "OUT" -- last event was OUT and shift end has passed
  - Red dot + "Not Arrived" -- no events today, expected in
  - Blue dot + "On Leave" -- has leave_record for today
- Shows today's first-in time, break duration, hours worked so far
- **Live camera feed widget**: OFF by default. "View Live Feed" button toggles it. Auto-off after 5 minutes with "Still watching?" prompt. Snapshot fallback (`GET /api/stream/snapshot`).
- Last event ticker -- scrolling feed of recent IN/OUT events via SSE with auto-reconnect
- Mobile: stacked layout. Feed and ticker are collapsible sections.

## 10.4 Attendance Log Page

- Full event history across all employees
- Filters: by person name, date range, department, event type (IN/OUT), manual-only
- Table: responsive card layout on mobile, table on desktop
- Columns: Person, Department, Event Type, Timestamp, Confidence
- Expandable row -- shows face snapshot thumbnail
- **Manual correction buttons**: Add Event, Edit, Delete (with confirmation dialog)

## 10.5 Employees Page

- Card grid (1 col mobile, 2 tablet, 3 desktop) with face photo, name, department

- **Add Employee** button --> full-screen modal on mobile:
  - `<input type="file" accept="video/*,image/*" capture="user">` for mobile camera capture
  - Upload progress bar via axios `onUploadProgress`
  - Fields: Full Name, Employee ID, Department, Job Title, Phone
  - Processing progress via polling endpoint
  - Quality gate feedback: "Photo too blurry", "Multiple faces detected", etc.
- **Edit Employee** -- update any field, re-enroll face if needed
- **Delete Employee** -- soft delete with confirmation dialog, records preserved
- Click any employee --> individual attendance history with calendar view

## 10.6 Unknowns Page

- Grid of all Unknown N persons with their auto-captured face snapshot
- Shows: number of visits, first seen, last seen
- **Rename button** --> modal with name, employee ID, department, job title
- On save: persons record updated, all past events retroactively linked, audit log entry
- Option to mark as "Visitor" (excluded from employee reports)

## 10.7 Reports Page

- Date range picker: native `<input type="date">` on mobile, custom calendar on desktop
- Preset buttons: "Today", "This Week", "This Month", "Last Month"
- Summary table: employee, days present, total hours, avg arrival, late days, absences
- Unknown visitors excluded from summary by default (toggle to include)
- Leave and holidays shown correctly (not counted as absences)
- **Download buttons:**
  - Download as Excel (`.xlsx`)
  - Download as PDF
  - Download Today's Report (partial, on demand)
- Report history: list of all past auto-generated reports with download links
- Large reports (500+ employees): generated asynchronously, SSE notification when ready

## 10.8 Leave Management Page (NEW)

- Calendar view showing leave records for all employees
- Add leave: select employee, date(s), leave type, optional note
- Manage holidays: add/edit/delete company holidays
- Configure weekend days (default: Saturday, Sunday)

## 10.9 Settings Page

- **Company Info** -- edit company name
- **Working Hours** -- shift start, shift end, late threshold, weekend days
- **Direction Calibration** -- recalibrate IN/OUT arrows anytime (with Pointer Events)
- **WiFi Settings** -- change SSID and password, switch WiFi mode
- **Admin Password** -- change admin credentials
- **Face Recognition** -- similarity threshold slider (0.4-0.8)

- **Data Retention** -- snapshot retention days, report retention days
- **System Status** -- uptime, DB size, disk space, camera status, CPU temp, last detection
- **Backup** -- trigger manual backup, download backup file
- **Restart Services** -- restart face engine or web server (via helper script)
- **Factory Reset** -- requires password re-entry + 30-second confirm

---

# 11. Backend API Endpoints

## 11.1 Employee Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/employees` | List all active employees |
| POST | `/api/employees` | Enroll new employee (multipart: photo/video + metadata) |
| GET | `/api/employees/{id}` | Get single employee + attendance stats |
| PUT | `/api/employees/{id}` | Update employee details |
| DELETE | `/api/employees/{id}` | Soft-delete employee (audit logged) |
| GET | `/api/employees/{id}/history` | Full attendance history |
| POST | `/api/employees/{id}/reenroll` | Re-enroll face |
| GET | `/api/employees/{id}/enrollment-status` | Poll enrollment processing progress |

## 11.2 Attendance Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/attendance/today` | All events today with current status per person |
| GET | `/api/attendance/events` | Paginated event log (filters: person_id, date, type) |
| POST | `/api/attendance/events` | **Manual event entry** (audit logged) |
| PUT | `/api/attendance/events/{id}` | **Correct an event** (audit logged) |
| DELETE | `/api/attendance/events/{id}` | **Delete erroneous event** (audit logged) |
| GET | `/api/attendance/summary` | Daily summaries (filters: date_from, date_to, person_id) |
| GET | `/api/attendance/live` | SSE stream of real-time detection events |

## 11.3 Unknown Persons Endpoints

| Method | Endpoint | Description |
|---|---|---|

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/unknowns` | List all unknown persons with stats |
| PUT | `/api/unknowns/{id}/rename` | Rename unknown (audit logged) |
| DELETE | `/api/unknowns/{id}` | Remove unknown person record |

## 11.4 Reports Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/reports/daily/{date}` | Get or generate report for specific date |
| GET | `/api/reports/history` | List all stored report files |
| GET | `/api/reports/download/{date}/excel` | Download Excel report (date validated: YYYY-MM-DD regex) |
| GET | `/api/reports/download/{date}/pdf` | Download PDF report (path traversal protected) |
| POST | `/api/reports/generate` | Manually trigger report generation (async, returns 202) |

## 11.5 Leave & Holiday Endpoints (NEW)

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/leave` | List leave records (filters: person_id, date_from, date_to) |
| POST | `/api/leave` | Add leave record |
| DELETE | `/api/leave/{id}` | Remove leave record |
| GET | `/api/holidays` | List all holidays |
| POST | `/api/holidays` | Add holiday |
| DELETE | `/api/holidays/{id}` | Remove holiday |

## 11.6 System Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/settings` | Get all system settings (excludes auth data) |
| PUT | `/api/settings` | Update settings (audit logged) |
| POST | `/api/calibration/set` | Save IN/OUT vectors |
| GET | `/api/calibration/get` | Get current vectors |
| GET | `/api/stream/video` | MJPEG stream (auth required, max 5 concurrent) |
| GET | `/api/stream/snapshot` | Single JPEG frame (lightweight alternative) |

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/system/status` | Camera, DB size, disk space, CPU temp, uptime |
| GET | `/api/system/health` | Health check for all 3 services |
| POST | `/api/system/restart/{service}` | Restart via helper script (piface-engine, piface-leds only) |
| POST | `/api/backup/create` | Trigger database backup |
| GET | `/api/backup/download` | Download latest backup |
| POST | `/api/auth/login` | Admin login (rate limited) |
| POST | `/api/auth/logout` | Invalidate session |
| POST | `/api/system/factory-reset` | Factory reset (requires password + confirmation) |

All endpoints return consistent JSON: `{success: bool, data: any, error: string|null}` All state-changing endpoints require CSRF token header. Report download endpoints validate date parameter with `^\d{4}-\d{2}-\d{2}$` regex and verify resolved path stays within reports directory.

---

# 12. Auto-Generated Daily Reports

## 12.1 Schedule

- Every day at `23:59:00` via APScheduler (NOT at midnight to avoid date boundary issues)
- On boot: check `MAX(date)` from `daily_summary`. Generate reports for all missing days.
- For days when Pi was off (no events): generate report showing all as ABSENT (minus leave/holidays)
- Reports saved to `reports/YYYY-MM-DD/attendance_YYYY-MM-DD.xlsx` and `.pdf`
- Staggered: daily_summary computed first, then reports generated 1 minute later

## 12.2 Report Generation

- Run in a **background subprocess** with `nice +10` (lower priority than face engine)
- Excel via `openpyxl`: Summary sheet + Full Log sheet + Unknowns sheet
- PDF via `reportlab`: Company header, summary table, color-coded status
- Leave and holidays clearly marked in both formats
- On-demand reports for current day return HTTP 202, generate async, notify via SSE

## 12.3 Data Retention

- Configurable retention period for reports (default: 365 days)
- Configurable retention for unknown person snapshots (default: 90 days)
- Cleanup job runs daily after report generation
- Disk space check: warn via dashboard + LED pattern when free space < 1GB

---

# 13. Pi Setup -- `install.sh`

Run **once** on a fresh Raspberry Pi OS 64-bit (Bookworm). Script has full error handling.

## 13.1 Script Structure

```bash
#!/bin/bash
set -euo pipefail
trap 'echo "INSTALL FAILED at line $LINENO. Check /var/log/piface-install.log"; exit 1' ERR

LOG="/var/log/piface-install.log"
exec > >(tee -a "$LOG") 2>&1

echo "=== PiFace Install Script v2.0 ==="
echo "Started at $(date)"
```

## 13.2 Steps in Order

```
1.  System update: apt update && apt upgrade -y

2.  Install system packages:
      hostapd dnsmasq nginx python3-pip python3-venv python3-dev
      build-essential cmake libopenblas-dev i2c-tools
      libcap-dev  # for gpiozero/lgpio

3.  Tell NetworkManager to ignore wlan0:
      Write /etc/NetworkManager/conf.d/piface.conf:
        [keyfile]
        unmanaged-devices=interface-name:wlan0
      systemctl restart NetworkManager

4.  Disable systemd-resolved (conflicts with dnsmasq on port 53):
      systemctl disable --now systemd-resolved
      rm -f /etc/resolv.conf
      echo "nameserver 127.0.0.1" > /etc/resolv.conf

5.  Configure static IP for wlan0 via systemd-networkd:
      Write /etc/systemd/network/10-wlan0.network
      systemctl enable systemd-networkd

6.  Write hostapd.conf (CCMP only, 802.11n, ACS, ap_isolate)

7.  Write dnsmasq.conf (1h lease, expanded range)

8.  Enable hostapd and dnsmasq services

9.  Create piface system user:
      useradd -r -s /usr/sbin/nologin -d /opt/piface piface
      usermod -aG gpio,video,i2c piface
```

```
10. Create directory structure:
      mkdir -p
/opt/piface/{database,snapshots,reports,backups,models,certs,config,helpers
}
      mkdir -p /var/log/piface
      chown -R piface:piface /opt/piface /var/log/piface

11. Create Python virtual environment at /opt/piface/venv:
      python3 -m venv /opt/piface/venv

12. Install Python dependencies (with piwheels for ARM64):
      /opt/piface/venv/bin/pip install --extra-index-url
https://www.piwheels.org/simple -r requirements.txt

13. Copy pre-bundled InsightFace Buffalo_S model files to models/
      (DO NOT download at runtime -- model files are in the repo)

14. Generate self-signed TLS certificate:
      openssl req -x509 -nodes -days 3650 -newkey rsa:2048 \
        -keyout /opt/piface/certs/piface.key \
        -out /opt/piface/certs/piface.crt \
        -subj "/CN=192.168.4.1"
      chmod 400 /opt/piface/certs/piface.key
      chown piface:piface /opt/piface/certs/*

15. Generate JWT secret:
      python3 -c "import secrets; print(secrets.token_hex(32))" >
/opt/piface/config/jwt_secret.key
      chmod 400 /opt/piface/config/jwt_secret.key
      chown piface:piface /opt/piface/config/jwt_secret.key

16. Build React frontend (or copy pre-built dist/):
      cd /opt/piface/frontend && npm ci && npm run build

17. Configure Nginx (HTTPS, reverse proxy, security headers, gzip, captive
portal)

18. Install systemd service files (with User=piface, watchdog, CPU
affinity)

19. Configure iptables firewall rules and persist with iptables-persistent

20. Configure logrotate for /var/log/piface/

21. Configure DS3231 RTC:
      echo "dtoverlay=i2c-rtc,ds3231" >> /boot/firmware/config.txt

22. Performance tuning:
      echo "performance" >
/sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
      echo "usb_max_current_enable=1" >> /boot/firmware/config.txt

23. Enable overlay filesystem for read-only root:
```

```
      raspi-config nonint enable_overlayfs

24. Set system timezone to Asia/Kolkata (configurable)

25. Configure Pi to boot to CLI (no desktop)

26. Disable unnecessary services:
      systemctl disable bluetooth hciuart triggerhappy
      systemctl mask ModemManager

27. Enable all PiFace services:
      systemctl enable piface-engine piface-web piface-leds

28. Set /etc/fstab: add noatime to root partition

29. Reboot
```

## 13.3 systemd Service Files

```
# piface-leds.service
[Unit]
Description=PiFace LED Controller
After=multi-user.target

[Service]
Type=notify
ExecStartPre=-/bin/rm -f /run/piface/leds.sock
ExecStart=/opt/piface/venv/bin/python /opt/piface/core/led_controller.py
Restart=always
RestartSec=3
User=piface
Group=piface
SupplementaryGroups=gpio
RuntimeDirectory=piface
WatchdogSec=30
TimeoutStopSec=10
CPUAffinity=3
MemoryMax=512M
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/run/piface /var/log/piface

[Install]
WantedBy=multi-user.target

# ---

# piface-engine.service
[Unit]
Description=PiFace Face Engine
```

```
After=piface-leds.service
Wants=piface-leds.service

[Service]
Type=notify
ExecStart=/opt/piface/venv/bin/python /opt/piface/core/face_engine.py
Restart=always
RestartSec=5
User=piface
Group=piface
SupplementaryGroups=gpio video
WatchdogSec=30
TimeoutStopSec=15
CPUAffinity=0 1
Nice=-5
MemoryMax=3G
Environment=OMP_NUM_THREADS=2
Environment=OPENBLAS_NUM_THREADS=2
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/opt/piface/database /opt/piface/snapshots /var/log/piface
/run/piface

[Install]
WantedBy=multi-user.target

# ---

# piface-web.service
[Unit]
Description=PiFace Web Server
After=network.target

[Service]
Type=notify
ExecStart=/opt/piface/venv/bin/uvicorn backend.main:app --uds
/run/piface/web.sock
Restart=always
RestartSec=5
User=piface
Group=piface
WatchdogSec=30
TimeoutStopSec=10
CPUAffinity=2
MemoryMax=1G
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/opt/piface/database /opt/piface/reports
/opt/piface/snapshots /opt/piface/backups /var/log/piface /run/piface

[Install]
WantedBy=multi-user.target
```

---

# 14. Build Phases for Claude Code

## Phase 0 -- Platform Foundation

- `setup/install.sh` with `set -euo pipefail`, error handling, logging
- NetworkManager wlan0 unmanage + systemd-networkd static IP
- systemd-resolved disable for dnsmasq
- `piface` user creation with gpio/video/i2c groups
- Self-signed TLS certificate generation
- JWT secret generation
- iptables firewall rules
- logrotate configuration
- DS3231 RTC setup
- CPU governor, USB power config
- All systemd service files (with User, Watchdog, CPUAffinity, MemoryMax)

## Phase 1 -- Core Infrastructure

- `backend/database.py` -- SQLAlchemy with WAL mode, FK enforcement, all PRAGMAs, migration system
- `backend/models.py` -- All tables with constraints, indexes, CHECK constraints
- `backend/main.py` -- FastAPI with HTTPS-only, CORS, CSRF, rate limiting, auth middleware
- `backend/security.py` -- JWT, bcrypt, CSRF tokens, rate limiter
- `core/led_controller.py` -- gpiozero, socket cleanup, sd_notify, SIGTERM handler
- `core/event_bus.py` -- Unix domain socket IPC with retry-on-connect

## Phase 2 -- Face Engine

- `core/camera.py` -- USB auto-detect, picamera2 fallback, CLAHE, buffer=1, reconnect loop
- `core/preprocessing.py` -- CLAHE, blur detection, enrollment quality gates
- `core/face_engine.py` -- InsightFace init, frame skip strategy (every 3rd), threshold 0.6, margin check, sd_notify, SIGTERM handler, state reconstruction on startup
- `core/tracker.py` -- IoU-based multi-object tracking, time-windowed direction detection, relative movement threshold
- `backend/routes/employees.py` -- enrollment with quality gates, progress polling, REFRESH signal

## Phase 3 -- Attendance System

- `backend/routes/attendance.py` -- event-log model, manual CRUD, cross-midnight handling, SSE live stream with auto-reconnect support
- Daily summary computation (derived breaks, not rigid states)
- State reconstruction from DB on engine restart
- Cooldown logic (45s per person per direction)

## Phase 4 -- Web Dashboard

- React scaffold: mobile-first, lazy routes, auth context, connection banner
- `Setup/` wizard with Pointer Events calibration (touch + mouse)
- `Today.jsx` -- responsive, MJPEG off by default, SSE ticker
- `Employees.jsx` -- mobile camera capture, upload progress, quality feedback
- `Unknowns.jsx` -- grid, rename modal, visitor flag
- `AttendanceLog.jsx` -- responsive table/cards, manual correction UI
- `Settings.jsx` -- all settings with confirmation dialogs

## Phase 5 -- Reports & Leave

- `core/report_generator.py` -- subprocess with nice +10, openpyxl + reportlab
- `backend/routes/reports.py` -- async generation (202 + SSE), download with path validation
- `Reports.jsx` -- date picker, presets, download buttons, report history
- `backend/routes/leave.py` -- leave records + holidays CRUD
- `Leave.jsx` -- calendar view, holiday management
- Data retention cleanup job

## Phase 6 -- Hardening

- Database backup system (nightly + manual + download)
- Disk space monitoring with LED warning
- Thermal monitoring with adaptive frame skipping
- Memory leak detection (RSS monitoring, MemoryMax)
- Nginx 502 -> "System starting up" page
- Missing-day report backfill (deferred 60s after boot)
- Embedding cache invalidation signal
- Camera reconnect with LED feedback
- Audit log for all admin actions

---

# 15. Critical Rules for Claude Code

## MUST FOLLOW -- No Exceptions

1. Never use cloud APIs, external services, or internet calls anywhere in the codebase
2. All three processes must be completely independent. LED works without web+engine. Engine works without web. Web works without engine.
3. SQLite is the only database. WAL mode mandatory. FK enforcement mandatory.
4. All Python runs in `/opt/piface/venv`. All services run as `piface` user, NEVER root.
5. All paths must be absolute in production code.
6. Face embeddings cached in numpy matrix in face engine. Never query DB per frame.
7. Camera stream: MJPEG over HTTPS. No WebRTC, no WebSockets for video.
8. Admin password stored as bcrypt hash in `auth_users` table (NOT in system_settings).
9. **Use gpiozero for GPIO. RPi.GPIO is broken on Pi 5.**
10. **Use NetworkManager unmanage + systemd-networkd. dhcpcd does not exist on Bookworm.**
11. **Process every 3rd frame for detection. Use centroid tracking between frames.**
12. **All uploads use UUID filenames. Never user-supplied filenames.**

13. **Uvicorn binds to Unix socket only. Never 0.0.0.0.**

## Architecture Decisions -- Do Not Change

1. InsightFace Buffalo_S model (pre-bundled, not downloaded)
2. Cosine similarity for face matching (threshold 0.6) and direction detection (threshold 0.7)
3. IoU-based multi-object tracking with persistent track IDs
4. Unix domain socket for LED event bus
5. APScheduler for midnight report generation
6. MJPEG stream endpoint for live video (max 5 concurrent clients)
7. React frontend served by Nginx over HTTPS
8. hostapd + dnsmasq for hotspot (NM unmanages wlan0)
9. systemd for process management with WatchdogSec
10. JWT in httpOnly + Secure + SameSite=Strict cookie
11. Event-log attendance model (IN/OUT events, derived summaries)
12. CLAHE preprocessing before face detection
13. Self-signed TLS certificate
14. iptables firewall (whitelist ports 80, 443, 53, 67-68 only)

## Quality Standards

1. All API endpoints return consistent JSON: `{success: bool, data: any, error: string|null}`
2. All database operations wrapped in try/except with proper rollback
3. All admin actions logged in `audit_log` table
4. Face engine logs to `/var/log/piface/engine.log` with RotatingFileHandler (10MB, 5 backups)
5. LED patterns must be non-blocking (gpiozero's built-in threading)
6. Enrollment video processing shows progress via polling endpoint
7. Unknown counter uses DB-level atomic increment
8. Calibration vectors stored as JSON. Both IN and OUT required before engine tracks.
9. React components handle loading (skeleton), error (retry button), and empty states
10. All file uploads validate via magic bytes AND file size before processing
11. Setup wizard enforced by middleware on every request when `setup_complete=false`
12. All systemd services send sd_notify watchdog pings
13. Report downloads validate date parameter with regex, verify resolved path

---

# 16. Complete Dependency Lists

## 16.1 Python `requirements.txt`

```
# Face Recognition
insightface==0.7.3
onnxruntime>=1.16.0
opencv-python-headless==4.8.1.78
scipy>=1.11.0
```

```
# Web Framework
fastapi==0.104.1
uvicorn[standard]==0.24.0
python-multipart==0.0.6
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4

# Database
sqlalchemy==2.0.23

# GPIO (Pi 5 compatible)
gpiozero>=2.0

# Reports
pandas==2.1.3
openpyxl==3.1.2
reportlab==4.0.7
Pillow==10.1.0

# Scheduling
APScheduler==3.10.4

# Security
python-magic==0.4.27
cryptography>=41.0.0

# Watchdog
sdnotify==0.3.2

# Utilities
numpy==1.26.2
pydantic==2.5.2
python-dotenv==1.0.0
```

**Install command:** `pip install --extra-index-url https://www.piwheels.org/simple -r requirements.txt`

## 16.2 Frontend `package.json`

```
{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0",
    "axios": "^1.6.2",
    "date-fns": "^2.30.0",
    "@tanstack/react-query": "^5.8.4",
    "lucide-react": "^0.294.0",
    "recharts": "^2.10.1"
  },
  "devDependencies": {
```

```
    "vite": "^5.0.0",
    "tailwindcss": "^3.3.5",
    "@vitejs/plugin-react": "^4.2.0",
    "vite-plugin-pwa": "^0.17.0"
  }
}
```

**Build optimization:** Configure Vite manual chunks to separate recharts. Tree-shake date-fns and lucide-react via named imports only.

### 16.3 System Packages (apt)

```
hostapd dnsmasq nginx
python3-pip python3-venv python3-dev
build-essential cmake
libopenblas-dev libcap-dev
i2c-tools
iptables-persistent
```

**Not needed:** ffmpeg (OpenCV bundles its own), nodejs/npm (pre-build frontend on dev machine)

---

## 17. Final Build Checklist

| # | Check | How to Verify |
|---|---|---|
| 1 | Active cooler fan spins on boot | Visual + CPU temp stays under 65C under load |
| 2 | Pi boots and hotspot appears within 30s | Connect to AttendanceSystem WiFi |
| 3 | HTTPS works (cert warning expected) | Browser to https://192.168.4.1 |
| 4 | HTTP redirects to HTTPS | Browser to http://192.168.4.1 |
| 5 | Port 8000 blocked by firewall | `curl http://192.168.4.1:8000` fails |
| 6 | SSH blocked by firewall | `ssh pi@192.168.4.1` fails |
| 7 | 5 green LED blinks on boot (gpiozero) | Watch LEDs on power-on |
| 8 | Setup wizard on first boot | Complete all 4 steps |
| 9 | Calibration works on phone touch | Draw arrows on phone screen |
| 10 | Enrollment rejects blurry photo | Upload blurry photo, see quality error |
| 11 | Enrollment with good video works | Upload 5s video, confirm recognition |
| 12 | Known person triggers green LED (IN) | Walk toward camera |
| 13 | Known person triggers red LED (OUT) | Walk away from camera |

| # | Check | How to Verify |
|---|---|---|
| 14 | Two people entering simultaneously | Both correctly identified |
| 15 | Unknown auto-named, snapshot saved | Walk unknown person past camera |
| 16 | Unknown rename updates all records | Rename from Unknowns page, check log |
| 17 | Manual attendance correction works | Add/edit/delete an event |
| 18 | Night shift cross-midnight works | IN before midnight, OUT after midnight |
| 19 | Today page shows live status (mobile) | Check on phone browser |
| 20 | Reports download Excel + PDF | Click download buttons |
| 21 | Leave record excludes from ABSENT | Add leave, verify report |
| 22 | Daily report auto-generates at 23:59 | Wait or check next morning |
| 23 | Camera disconnect triggers reconnection | Unplug USB camera, wait, replug |
| 24 | Watchdog restarts hung process | `kill -STOP <pid>`, verify restart |
| 25 | Power loss recovery (pull plug) | Pull power, verify clean boot + DB intact |
| 26 | Database backup works | Trigger from Settings, download |
| 27 | Rate limiting blocks brute-force | 6+ wrong passwords, verify lockout |
| 28 | RTC keeps time after reboot | Reboot, verify accurate time |
| 29 | WiFi SSID/password change works | Change from Settings, reconnect |
| 30 | Factory reset works (with confirmation) | Trigger reset, verify setup wizard returns |

---

*PiFace Attendance System -- Build Specification v2.0 One Pi. One Camera. Two LEDs. Zero Internet. Runs Forever. Now hardened for the real world.*